

ISIT312 Big Data Management

Spark Operations

Dr Guoxin Su and Dr Janusz R. Getta

School of Computing and Information Technology -
University of Wollongong

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

[Operations on DataFrames](#)

[SQL Module](#)

Quick Start

A `SparkSession` instance is an entry to a `Spark` application

If you type `spark` in the spark-shell interface then you get the following messages

```
spark
<pyspark.sql.session.SparkSession object at 0x726c633ca740>
```

Message

You can use `SparkSession` instance `spark` to interact with `Spark` and to develop your data processing pipeline

For example,

```
range1000 = spark.range(1000).toDF("number")
```

Creating a DataFrame

```
range1000.show(2)
```

```
+-----+
```

```
|number|
```

```
+-----+
```

```
|      0|
```

```
|      1|
```

```
+-----+
```

```
only showing top 2 rows
```

listing a DataFrame

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

4/34

Quick Start

Sample processing of a file `README.txt`

```
lines = sc.textFile("https://url.au.m.mimecastprotect.com/s/jrFwCYW8EEtLDoj2lsVs1FxWIoc?domain=127.0.0.1")
```

Creating RDD

```
lines.first()
```

Reading the first line

```
'For the latest information about Hadoop, please visit our website at:'
```

Results

```
lines.count()
```

Counting lines

```
7
```

Results

```
lines.filter(lambda x: "apache" in x).collect()
```

Filtering rows

```
[' https://url.au.m.mimecastprotect.com/s/-10FCZY166h57nX9ghxtDFBUwAc?domain=hadoop.apache.org/', ' https://url.au.m.mimecastprotect.com/s/-10FCZY166h57nX9ghxtDFBUwAc?domain=hadoop.apache.org/']
```

Results

Quick Start

More operations on RDD

Creating RDD

```
lines = sc.textFile("https://url.au.m.mimecastprotect.com/s/jrFwCYW8EEtLDoj2lsVs1FxWIoc?domain=127.0.0.1")
```

Listing RDD

```
lines.collect()
['For the latest information about Hadoop, please visit our website at:', ' ', 'https://url.au.m.mimecastpr
```

Creating words and counting

```
counters = lines.flatMap(lambda x: x.split()).countByValue()
```

Listing words and counters

```
for i, (word, count) in enumerate(counters.items()):
...     print(word, count)
```

Results

```
For 1
the 1
latest 1
information 1
about 1
Hadoop, 1
please 1
visit 1
our 2
... ..
```

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

6/34

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

[Operations on DataFrames](#)

[SQL Module](#)

Self-Contained Application

A sample self-contained application `SimpleApp.py`

```
"""SimpleApp.py"""
from pyspark.sql import SparkSession

logFile = "README.txt" # In home folder https://url.au.m.mimecastprotect.com/s/1KaGC2xMkktVo0z3sXCEF5Nag3?don
spark = SparkSession.builder.appName("SimpleApp").getOrCreate()

logData = spark.read.text(logFile).cache()
numAs = logData.filter(logData.value.contains('a')).count()
numBs = logData.filter(logData.value.contains('b')).count()
print("Lines with a: %i, lines with b: %i" % (numAs, numBs))
spark.stop()
```

SimpleApp.py

Processing of an application `SimpleApp.py`

```
$SPARK_HOME/bin/spark-submit --master local[4] SimpleApp.py
```

Processing

```
Lines with a: 4, lines with b: 1
```

Results

```
hdfs dfs -cat /user/bigdata/README.txt
```

Listing README.txt

```
For the latest information about Hadoop, please visit our website at:
```

Listing README.txt

```
https://url.au.m.mimecastprotect.com/s/-10FCZY166h57nX9ghxtDFBUwAc?domain=hadoop.apache.org/
```

```
and our wiki, at:
```

```
https://url.au.m.mimecastprotect.com/s/m2IfC1WLwwtM6wLNGF1u0FVI5lF?domain=cwiki.apache.org/
```

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

8/34

Self-Contained Application

Another sample self-contained application `AnotherApp.py`

```
"""AnotherApp.py"""
from pyspark import SparkConf, SparkContext

spark_conf = SparkConf()
sc = SparkContext(conf=spark_conf)

lines = sc.textFile("README.txt") # In home folder https://url.au.m.mimecastprotect.com/s/1KaGC2xMkktPVo0z3sX(
lineLengths = lines.map(lambda s: len(s))
totalLength = lineLengths.reduce(lambda a, b: a + b)
print(totalLength)

sc.stop()
```

`AnotherApp.py`

Processing of an application `AnotherApp.py`

```
$SPARK_HOME/bin/spark-submit --master local[4] AnotherApp.py
```

`Processing`

```
168
```

`Results`

```
hdfs dfs -cat /user/bigdata/README.txt
```

`Listing README.txt`

```
For the latest information about Hadoop, please visit our website at:
```

`Listing README.txt`

```
https://url.au.m.mimecastprotect.com/s/-10FCZY166h57nX9ghxtDFBUwAc?domain=hadoop.apache.org/
```

```
and our wiki, at:
```

```
https://url.au.m.mimecastprotect.com/s/m2IfC1WLwwtM6wLNGF1u0FVI5lF?domain=cwiki.apache.org/
```

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

9/34

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

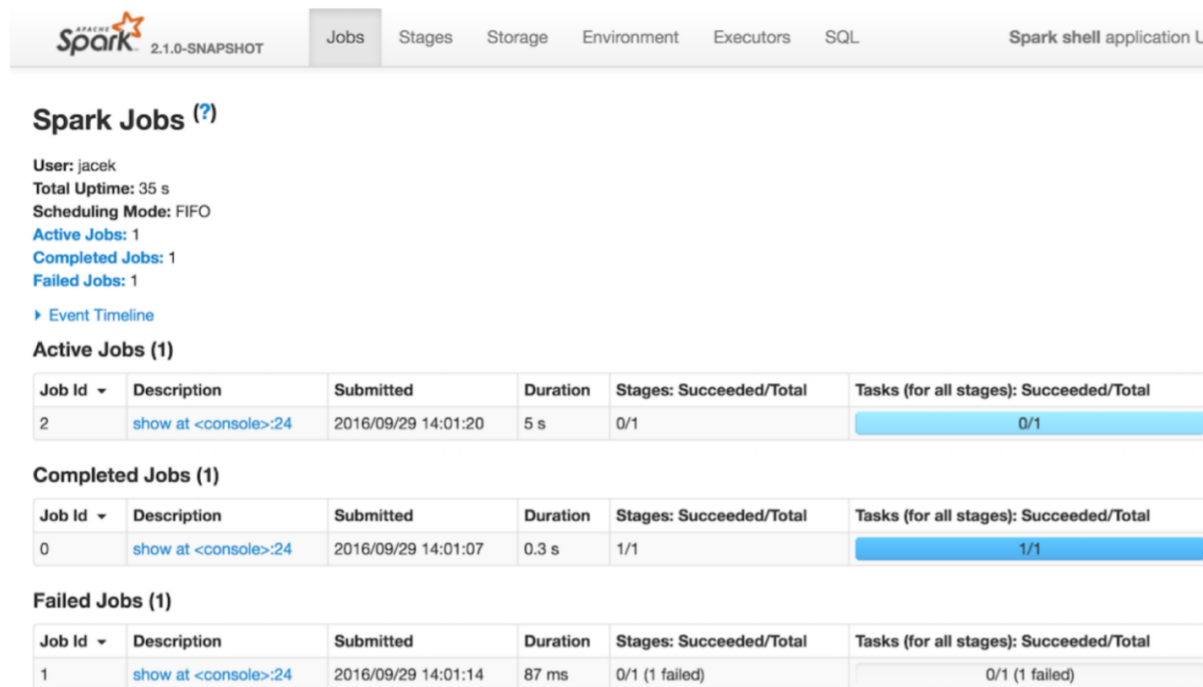
[Operations on DataFrames](#)

[SQL Module](#)

Web User Interface

Each driver program has a Web UI, typically on port **4040**

Spark Web UI displays information about running tasks, executors, and storage usage.



Spark Jobs (?)

User: jacek
Total Uptime: 35 s
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 1
Failed Jobs: 1
[Event Timeline](#)

Active Jobs (1)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
2	show at <console>:24	2016/09/29 14:01:20	5 s	0/1	0/1

Completed Jobs (1)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0	show at <console>:24	2016/09/29 14:01:07	0.3 s	1/1	1/1

Failed Jobs (1)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
1	show at <console>:24	2016/09/29 14:01:14	87 ms	0/1 (1 failed)	0/1 (1 failed)

In HTML view press 'p' to see the [lecture notes](#)
[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

11/34

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

[Operations on DataFrames](#)

[SQL Module](#)

Operations on Resilient Distributed Datasets (RDDs)

Operations on **RDDs** are performed on raw **Java** or **Scala** objects

Creating a simple **RDD** with words

```
hdfs dfs -cat /user/bigdata/README.txt
```

Listing a file in HDFS

Creating lines and words

```
lines = sc.textFile("README.txt")
words = lines.flatMap(lambda line: line.split(' '))
```

Creating RDDs

Eliminating duplicates and counting words

```
distinct_words = words.distinct()
distinct_words.count()
```

Distinct and counting

Filtering

```
a_words = words.filter(lambda word: word.startswith("a"))
```

Filtering words starting with "a"

[In HTML view](#) press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

13/34

Operations on Resilient Distributed Datasets (RDDs)

Sorting with `sortBy`

```
sc.parallelize(['a', 'd', 'c', 'e', 'b']).sortBy(lambda x: x).collect()
```

Sorting in ascending order

```
sc.parallelize(['a', 'd', 'c', 'e', 'b']).sortBy(lambda x: -ord(x)).collect()
```

Sorting in descending order

```
sc.parallelize([1, 2, 5, 3, 4]).sortBy(lambda x: -x).collect()
```

Sorting in descending order

```
sc.parallelize([1, 2, 5, 3, 4]).sortBy(lambda x: x).collect()
```

Sorting in ascending order

Sorting pairs

```
sc.parallelize([('1', 3), ('2', 5), ('a', 1), ('b', 2), ('d', 4)]).sortBy(lambda x: x[0]).collect()
```

Sorting pairs by first element

```
sc.parallelize([('1', 3), ('2', 5), ('a', 1), ('b', 2), ('d', 4)]).sortBy(lambda x: x[1]).collect()
```

Sorting pairs by second element

Operations on Resilient Distributed Datasets (RDDs)

Sorting with `sortByKey`

Sorting by the first key

```
sc.parallelize([('a', 1), ('b', 2), ('1', 3), ('d', 4), ('2', 5)]).sortByKey().first()  
('1', 3)
```

Sorting by the first key

```
sc.parallelize([('a', 1), ('b', 2), ('1', 3), ('d', 4), ('2', 5)]).sortByKey(True, 1).collect()  
[('1', 3), ('2', 5), ('a', 1), ('b', 2), ('d', 4)]
```

Sorting by the second key

```
sc.parallelize([('a', 1), ('b', 2), ('1', 3), ('d', 4), ('2', 5)]).sortByKey(True, 2).collect()  
[('1', 3), ('2', 5), ('a', 1), ('b', 2), ('d', 4)]
```

Extending RDD

```
tmp2 = [('Mary', 1), ('had', 2), ('a', 3), ('little', 4), ('lamb', 5)]  
tmp2.extend([('whose', 6), ('fleece', 7), ('was', 8), ('white', 9)])
```

Sorting with a function

```
sc.parallelize(tmp2).sortByKey(True, 3, keyfunc=lambda k: k.lower()).collect()  
[('a', 3), ('fleece', 7), ('had', 2), ('lamb', 5), ... ('white', 9), ('whose', 6)]
```

Operations on Resilient Distributed Datasets (RDDs)

Random split

```
rdd = sc.parallelize(range(500), 1)
rdd1, rdd2 = sc.parallelize(range(500), 1).randomSplit([2, 3], 17)
```

Random split

```
len(rdd1.collect() + rdd2.collect())
500
150 < rdd1.count() < 250
True
250 < rdd2.count() < 350
True
```

Results

Reduce RDD to one value

```
from operator import add
sc.parallelize([1, 2, 3, 4, 5]).reduce(add)
15
```

Reducing

```
sc.parallelize((2 for _ in range(10))).map(lambda x: 1).cache().reduce(add)
10
```

Reducing

Operations on Resilient Distributed Datasets (RDDs)

map versus flatMap

```
sc.parallelize(["a b c","c d e"]).map(lambda line: line.split(" ")).collect()
[['a', 'b', 'c'], ['c', 'd', 'e']]
```

map

```
sc.parallelize(["a b c","c d e"]).flatMap(lambda line: line.split(" ")).collect()
['a', 'b', 'c', 'c', 'd', 'e']
```

flatMap

Iteration operation foreach

```
def f(x):
    print(x)

sc.parallelize([1, 2, 3, 4, 5]).foreach(f)
```

foreach

isEmpty operation

```
sc.parallelize([]).isEmpty()
True
sc.parallelize(["a"]).isEmpty()
False
```

foreach

Operations on Resilient Distributed Datasets (RDDs)

reduce operation

```
from operator import add
sc.parallelize([1, 2, 3, 4, 5]).reduce(add)
15
```

reduce

```
sc.parallelize((2 for _ in range(10))).map(lambda x: 1).cache().reduce(add)
10
```

reduce

reduceByKey operation

```
from operator import add
sorted(sc.parallelize([("a", 1), ("b", 1), ("a", 1)]).reduceByKey(add).collect())
[('a', 2), ('b', 1)]
```

reduce

Operations on Resilient Distributed Datasets (RDDs)

groupBy operation

```
result = sc.parallelize([1, 1, 2, 3, 5, 8]).groupBy(lambda x: x % 2)
sorted([(x, sorted(y)) for (x, y) in result])
[(0, [2, 8]), (1, [1, 1, 3, 5])]
```

groupBy

groupByKey operation

```
sorted(sc.parallelize([("a", 1), ("b", 1), ("a", 1)]).groupByKey().mapValues(len).collect())
[('a', 2), ('b', 1)]
```

groupByKey

```
sorted(sc.parallelize([("a", 1), ("b", 1), ("a", 1)]).groupByKey().mapValues(list).collect())
[('a', [1, 1]), ('b', [1])]
```

groupByKey

join operation

```
rdd1 = sc.parallelize([("a", 1), ("b", 4)])
rdd2 = sc.parallelize([("a", 2), ("a", 3)])
sorted(rdd1.join(rdd2).collect())
[('a', (1, 2)), ('a', (1, 3))]
```

join

[In HTML view](#) [press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

19/34

Operations on Resilient Distributed Datasets (RDDs)

union operation

```
rdd = sc.parallelize([1, 1, 2, 3])  
rdd.union(rdd).collect()  
[1, 1, 2, 3, 1, 1, 2, 3]
```

union

intersection operation

```
rdd1 = sc.parallelize([1, 10, 2, 3, 4, 5])  
rdd2 = sc.parallelize([1, 6, 2, 3, 7, 8])  
rdd1.intersection(rdd2).collect()  
[2, 1, 3]
```

intersection

subtract operation

```
rdd1 = sc.parallelize([("a", 1), ("b", 4), ("b", 5), ("a", 3)])  
rdd2 = sc.parallelize([("a", 3), ("c", None)])  
sorted(rdd1.subtract(rdd2).collect())  
[('a', 1), ('b', 4), ('b', 5)]
```

subtract

[In HTML view](#) [press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

20/34

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

[Operations on DataFrames](#)

[SQL Module](#)

Operations on DataFrames

Spark DataFrame support two types of operations: **transformations** and **actions**

Transformations are operations on **DataFrames** that return a new **DataFrame**

- For example `select()`, `groupBy()`, `map()`, and `filter()`

Actions are operations that return a result to the driver program or write it to storage, and kick off a computation

- For example `show()`, `count()`, and `first()`

Return type difference: **transformations** return **DataFrames/Datasets**, whereas **actions** return some other data type

Spark treats the two operations very differently

Operations on DataFrames

Transformations are **lazily evaluated**, meaning that **Spark** will not begin to execute until it sees an action

Instead, **Spark** internally records metadata to indicate that some transformation operation has been requested

For example **transformation** creates another **DataFrame**

```
sqlDF = spark.sql("SELECT * FROM vpeople"))
```

Creating a DataFrame

Action triggers the computation

```
sqlDF.show()
```

Action on a DataFrame

Operations on DataFrames

The **lazy evaluation** to reduce the number of passes it has to take over the dataset

In **Hadoop MapReduce**, developers often have to consider how to group together operations to minimize the number of MapReduce passes

In Spark, there is no substantial benefit to writing a single complex map instead of chaining together many simple operations

Thus, users are free to organize their program into smaller, more manageable operations

Operations on DataFrames

DataFrame is a data abstraction for **Spark SQL**

Dataset is a distributed collection of data

- It supports the use of self-defined functions to process data
- For example, **map** and **reduce** functions in the previous slides
- **Dataset** is typed; typing is checked at compiling time

DataFrame is a **Dataset** organized into named columns.

- It is conceptually equivalent to a table in a relational database or a data frame in **R/Python**
- To use self-defined functions, you need to register them with **Spark**
- **DataFrame** is untyped, i.e., typing is checked at runtime
- **DataFrame** is more performance-optimal than **Dataset**

Operations on DataFrames

A **DataFrame** can be created in the following way

```
hdfs dfs -cat people.json
{"name":"James","age":25}
{"name":"Harry","age":24}
{"name":"Robin","age":50}
```

A JSON file in HDFS

Loading JSON file from HDFS and creating a DataFrame

```
df = spark.read.load("people.json", format="json")
```

Listing a data frame

```
>>> df.show()
+---+-----+
|age| name|
+---+-----+
| 25|James|
| 24|Harry|
| 50|Robin|
+---+-----+
```

Listing a schema

```
df.printSchema()
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
```

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

26/34

Operations on DataFrames

A **DataFrame** can be saved in **HDFS** in the following way

Saving a DataFrame as csv file

```
df.select("name").write.save("names", format="csv")
```

Results in folder names

```
hdfs dfs -ls names
```

Found 2 items

```
-rw-r--r--    1 bigdata supergroup      0 2026-04-20 13:59 names/_SUCCESS
-rw-r--r--    1 bigdata supergroup    18 2026-04-20 13:59 names/part-00000-f1e3483e-810b-4be6-8875-
                                         4587cb6be954-c000.csv
```

Results csv file

```
hdfs dfs -cat names/part*
```

James

Harry

Robin

Reading and writing a Parquet file

Creating a DataFrame from a Parquet file

```
df = spark.read.load("examples/src/main/resources/users.parquet")
```

Writing a DataFrame to a Parquet file

```
df.select("name", "favorite_color").write.save("namesAndFavColors.parquet")
```

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

27/34

Operations on DataFrames

Operations on **DataFrames** included in a class
`pyspark.sql.DataFrame`

```
people = spark.createDataFrame([  
    {"deptId": 1, "age": 40, "name": "Hyukjin Kwon", "gender": "M", "salary": 50},  
    {"deptId": 1, "age": 50, "name": "Takuya Ueshin", "gender": "M", "salary": 100},  
    {"deptId": 2, "age": 60, "name": "Xinrong Meng", "gender": "F", "salary": 150},  
    {"deptId": 3, "age": 20, "name": "Haejoon Lee", "gender": "M", "salary": 200}  
])
```

Creating a DataFrame

```
department = spark.createDataFrame([  
    {"id": 1, "name": "PySpark"},  
    {"id": 2, "name": "ML"},  
    {"id": 3, "name": "Spark SQL"}  
])
```

Creating a DataFrame

```
people.count()
```

Counting rows

```
df1 = people.filter(people.age > 50)  
df2 = people.where(df.age == 60)
```

Filtering by column instances

```
df3 = people.filter('age > 30')  
df4 = people.where('age = 60')
```

Filtering by SQL expression in a string

[In HTML view press 'p' to see the lecture notes](#)

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

28/34

Operations on DataFrames

Operations on **DataFrames** included in a class
pyspark.sql.DataFrame

```
df5 = people.select('*')
```

Projection

```
df6 = people.select(people.name, (people.age + 10).alias('age'))
```

Grouping, global aggregation

```
df7 = people.groupBy().avg()
```

Grouping, global aggregation

```
df8 = people.groupBy("age").count()
```

Grouping and specification of a dictionary to calculate the summation of 'age'

```
df9 = people.groupBy("name").agg({"age": "sum"}).sort("name")
```

Grouping and calculation of maximum values.

```
df10 = people.groupBy(people.name).max().sort("name")
```

Grouping by 'name' and 'age', and calculation of the number of rows in each group

```
df11 = people.groupBy(["name", people.age]).count().sort("name", "age")
```

Operations on DataFrames

Operations on **DataFrames** included in a class
`pyspark.sql.DataFrame`

Union

```
df12 = spark.createDataFrame([(1, 'A'), (2, 'B')], ['id', 'value'])
df13 = spark.createDataFrame([(3, 'C'), (4, 'D')], ['id', 'value'])
df14 = df12.union(df13)
```

Intersection

```
df15 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3), ("c", 4)], ["C1", "C2"])
df16 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3)], ["C1", "C2"])
df17 = df16.intersect(df15).sort(df16.C1.desc()).show()
```

Subtract(Difference)

```
df18 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3), ("c", 4)], ["C1", "C2"])
df19 = spark.createDataFrame([("a", 1), ("a", 1), ("b", 3)], ["C1", "C2"])
df20 = df18.subtract(df19).show()
```

Operations on DataFrames

Operations on **DataFrames** included in a class
pyspark.sql.DataFrame

Creating DataFrames

```
from pyspark.sql import Row
from pyspark.sql.functions import desc
df1 = spark.createDataFrame([(2, "Alice"), (5, "Bob")]).toDF("age", "name")
df2 = spark.createDataFrame([Row(height=80, name="Tom"), Row(height=85, name="Bob")])
df3 = spark.createDataFrame([Row(age=2, name="Alice"), Row(age=5, name="Bob")])
df4 = spark.createDataFrame([Row(age=10, height=80, name="Alice"),
                             Row(age=5, height=None, name="Bob"),
                             Row(age=None, height=None, name="Tom"),
                             Row(age=None, height=None, name=None) ])
```

Inner joins

```
df21 = df1.join(df2, 'name').select(df2.name, df2.height)
df22 = df1.join(df4, ['name', 'age']).select(df4.name, df4.age).show()
```

outer joins

```
df23 = df1.join(df2, df1.name == df2.name, 'outer')
        .select(df2.name, df2.height).sort(desc("name")).
df24 = df1.join(df2, 'name', 'outer').select('name', 'height').sort(desc("name")).show()
```

Spark Operations

Outline

[Quick Start](#)

[Self Contained Application](#)

[Web User Interface](#)

[Operations on Resilient Distributed Datasets \(RDDs\)](#)

[Operations on DataFrames](#)

[SQL Module](#)

SQL Module

Register a **DataFrame** as **SQL** temporary view

```
people.createOrReplaceTempView("vpeople")
```

Registering a temporary view

```
sqlDF = spark.sql("SELECT * FROM vpeople")  
sqlDF.show()
```

Using SQL

In HTML view press 'p' to see the lecture notes

[TOP](#)

Created by Janusz R. Getta, ISIT312/ISIT912 Big Data Management, Spring, 2026

33/34

References

[RDD Programming Guide](#)

[Spark SQL, DataFrames and Datasets Guide](#)

A Gentle Introduction to Spark, Databricks, (Available in **READINGS** folder)

Karau H., Fast data processing with Spark Packt Publishing, 2013
(Available from UOW Library)

Srinivasa, K.G., Guide to High Performance Distributed Computing: Case Studies with Hadoop, Scalding and SparkSpringer, 2015 (Available from UOW Library)

Chambers B., Zaharia M., Spark: The Definitive Guide, O'Reilly 2017

Perrin J-G., Spark in Action, 2nd ed., Manning Publications Co. 2020